

RedTeamForge: Architecture & Summary Report

1. Overview

This report details the work completed in the `Final/` directory for the **RedTeamForge** platform, an autonomous LLM red teaming and security testing framework. The current system encapsulates a FastAPI backend connected to a Vanilla JS/HTML/CSS dashboard and a PostgreSQL database. It seamlessly integrates local instances of Large Language Models (LLMs) via Ollama to autonomously generate testing probes, evaluate victim LLM responses, and intelligently mutate attack prompts to identify vulnerabilities.

2. Directory Structure and File Explanations

The `Final/` directory is logically partitioned into routers, schemas, services, static assets, and core server configurations. Here is a comprehensive breakdown of every file:

Core Configuration & Server Files

- **main.py**: The central entrypoint of the application. It initializes the FastAPI server instance, configures Cross-Origin Resource Sharing (CORS) middleware, and mounts directory routers (`main_router`). Additionally, it automatically mounts the `/static` directory and provides a root redirect (`/`) to the GUI dashboard (`index.html`).
- **config.py**: Defines application settings using Pydantic's `SettingsConfigDict`. Key configurations include the `DATABASE_URL` for PostgreSQL, the local `OLLAMA_URL` endpoint, default test model identities (`gemmaSecure:latest` as victim, `mistral-nemo:latest` as judge), and operational thresholds like `OLLAMA_TIMEOUT` and `MAX_SESSION_TIME_MINUTES`.
- **database.py**: Handles database connectivity via SQLAlchemy. It initializes a connection engine to the local PostgreSQL instance, configures connection pooling (`pool_size=10`), and implements a `get_db()` dependency injection to yield database sessions to route controllers safely.
- **requirements.txt**: Specifies the Python dependencies necessary to run the project.
- **debug.py & err.log**: Files used for tracking exceptions and isolated script-testing debugging.
- **out.txt**: Used for staging outputs or terminal log caching.
- **ReadMe Files (`README.md`, `ReadMe_Final.md` etc)**: Foundational documentation describing how to execute scripts and directory structures locally.

Routers (`routers/`)

This directory stores all API endpoint definitions.

- **main_router.py**: A centralized router aggregating the modular `probe_router` and `mutation_router`.
- **probe.py**: Exposes the `/probes/start` and `/probes/status` endpoints. It handles GUI requests to orchestrate automated probing. The `/start` endpoint invokes `probe_service.run_probe_session()` as a FastAPI `BackgroundTask`, allowing immediate response for asynchronous polling on `/status`.

- **mutation.py**: Provides the `/mutation/run` endpoint, invoking the `MutatorService` to generate complex variants of attack prompts based on previously discovered vulnerabilities.

Schemas (**schemas/**)

This module enforces data consistency via Pydantic models.

- **probe.py**: Holds state models like `ProbeResponse` and `ProbeSessionStatus`. These structure how probe telemetry (covered criteria, weak areas, timestamps) is sent to the dashboard.
- **scope.py**: Contains `ScopeCreate`, capturing the required `scope_text` and an optional `target_endpoint` indicating the target API testing surface.
- **mutation.py**: Represents the `MutationStatus` model to return successfully generated prompts and generation counts.
- **victim.py**: Schemas representing interactions or status specifically associated with standard victim target configurations.

Services (**services/**)

This layer handles the heavy business logic and autonomous LLM workflows.

- **probe_service.py**: Implements the `ProbeService` and `ProbeMemory` classes. It manages a complete autonomous testing loop:
 1. Reaching out to the attacker LLM to generate standard probe attacks tailored to the target scope.
 2. Forwarding the generated probe asynchronously to the user's `target_endpoint` webhook or a local default victim model.
 3. Asynchronously parsing the victim's response using the judge LLM to identify vulnerabilities and update the `ProbeMemory` cache with newfound "Covered Criteria" and "Weak Areas."
- **mutator_service.py**: Employs genetic-algorithm-inspired techniques. It:
 1. Selects top "parent prompts" from a PostgreSQL `genomes` table based on complexity and relevant attack techniques.
 2. Calls the LLM to mutate these parents into more advanced attacks, adapting to weaknesses discovered in the `ProbeMemory`.
 3. Persists newly generated prompts to a `generated_prompts` table for persistence and further lineage tracking. Fast background webhook invocation is included.
- **report_service.py**: Designed as a placeholder/skeleton to aggregate findings into post-session executive summaries.

Static Frontend (**static/**)

- **index.html**: A beautifully styled, glassmorphism-infused admin dashboard layout. Houses the layout for telemetry statistics, configuration input fields, and the live status feed.
- **style.css**: Advanced, rich CSS featuring grid layouts, animations (`@keyframes`), and vibrant thematic colors rendering the visually striking "Live Simulation Active" interface.
- **app.js**: Vanilla JS logic connecting the frontend to the FastAPI backend. Implements

fetching, state synchronization, interval status polling (`/probes/status`), and DOM manipulation to inject populated UI cards as new probes execute.

3. LLM Integration Pipeline

The system integrates LLMs completely locally, leveraging the Ollama API runtime.

1. **Orchestration API:** Python services (`requests.post()`) hit `http://localhost:11434/api/generate`.
2. **Model Segregation:** The system heavily segregates tasks between different loaded models. A "Judge/Attacker" model (`mistral-nemo:latest`) is tasked with intelligently deriving probing criteria, drafting prompts, and evaluating victim vulnerabilities. A "Victim" model (`gemmaSecure:latest`) is used when no external Webhook API target is provided.
3. **Autonomous Parsing Engine:** The output of one API LLM call dictates the exact prompt structure of the next. For instance, the `probe_service` feeds previous `victim_response` payloads directly back into a fresh Judge prompt to dynamically extract `key_insight` and update memory.
4. **Generative Mutation Array:** The `mutator_service` acts as an evolution engine, providing previous prompts (loaded directly from SQL DB via pandas dataframe) inside an LLM context window to demand a syntactically more complex permutation.
5. **Concurrency Optimization:** By utilizing Python's `asyncio.to_thread()`, slow synchronous requests to Ollama do not block the FastAPI event pool, ensuring the UI remains perfectly responsive and continues receiving dashboard metrics smoothly.

4. Future Scope

While functionally robust, several domains are established for immediate upscaling in future phases:

- **Target Interface Refinement:** While Custom Endpoint injection acts as a lightweight Webhook to pipe newly forged payloads out from the Red Teaming session to a target Application properly, support for Header Injection, REST verb selection, and JWT Authentication parameters for internal company testing should be designed inside a custom `Scope` schema.
- **Report Generation Automation:** Tying the `report_service.py` to compile final, exportable `.pdf` or `.docx` compliance reports once a session halts. Right now, live state sits transient inside the Event Loop unless manually recorded via Web UI observation.
- **Distributed Prompting & Background Tasks Processing:** As prompt array generation volume exponentially increases, queueing architectures (i.e. Celery, Redis) are optimally poised to handle aggressive mass-parallel testing and long-horizon mutations instead of relying solely on built-in FastAPI `BackgroundTasks`.
- **Docker Containerization:** Packaging the entirety of the `Final/` directory, along with a hardened Postgres instance, inside a unified `docker-compose.yaml` to ensure zero-friction cold platform spin-ups across developer workstations.
- **Custom Prompt Marketplaces / Genomes Integration:** Enhancing the database tabular relations to enable security analysts to interface with and populate `genomes` metadata completely natively, providing a playground for adversarial scenario templating.
- **Adaptive Persona Logic:** Structurally instructing the judge model LLM via prompt tags to autonomously shift identities and roles (e.g., highly technical backend dev vs

confused elderly end-user) when iterating on probes to exploit role-based constraints.